

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	BL4 Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	BL6 Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 Apply

7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 Ana yze
---	---	---------------

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 Eva uate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 Eva uate

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

NAME: MONIKA.B

REG. NO: 192571002

Q1 — A university stores student and course data in a single table. Analyze the structure, identify normalization issues, and redesign it up to 3NF.

1. The Original (Unnormalized) Table

University(student_id, student_name, dept_id, dept_name, advisor, course_id, course_name, credits, instructor, semester, grade)

A student can enrol in many courses, so course-related columns repeat for every course a student takes — this is a repeating group, which already violates 1NF.

2. Issues Identified

- Violation of 1NF: for a student enrolled in several courses, (course_id, course_name, credits, instructor, semester, grade) repeats once per course inside the same row/table — a multi-valued, non-atomic structure.
- Partial dependency (2NF violation): once the table is flattened with a composite key (student_id, course_id), course_name, credits and instructor depend only on course_id, not on the whole key — and student_name, dept_id, dept_name, advisor depend only on student_id.
- Transitive dependency (3NF violation): dept_name and advisor depend on dept_id, and dept_id depends on student_id — so dept_name depends on student_id transitively through dept_id, instead of depending directly on the key.
- Consequences: update anomaly (changing a department's name means updating it in every row of every student in that department), insertion anomaly (cannot record a new department until a student joins it), deletion anomaly (deleting the last student of a department loses the department's data entirely).

3. Step-by-step Normalization

1NF: Remove the repeating course group — create one row per (student_id, course_id) enrolment instead of embedding a list of courses inside a student row.

2NF: With the composite key (student_id, course_id) in the enrolment table, remove attributes that depend on only part of the key: student_name/dept_id/dept_name/advisor move to a Student table; course_name/credits/instructor move to a Course table. Only semester and grade stay with the composite key.

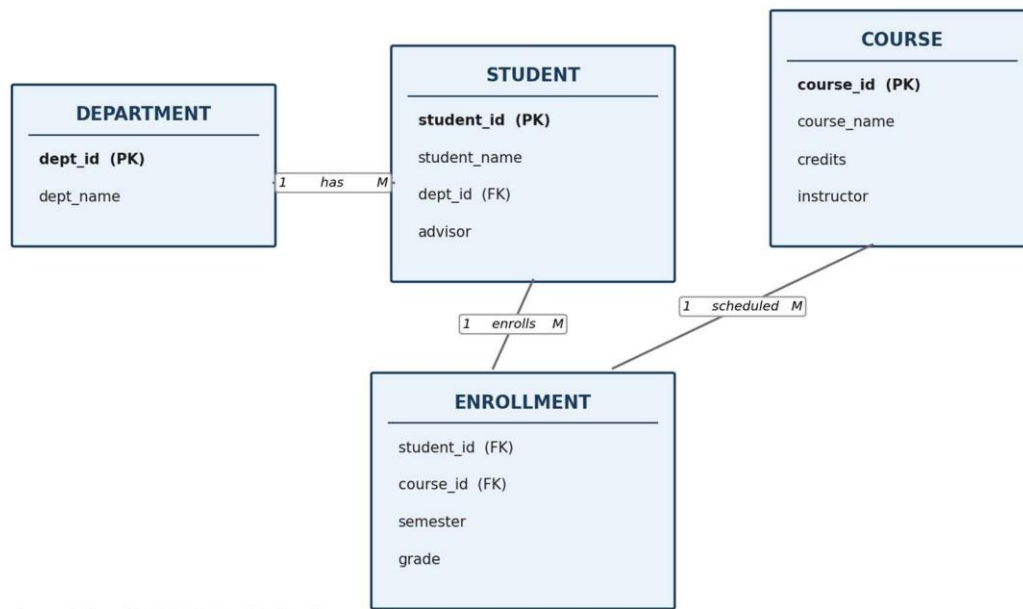
3NF: Remove the transitive dependency inside Student: dept_name and advisor depend on dept_id, not on student_id, so they move to a separate Department table, leaving Student(student_id, student_name, dept_id) referencing Department(dept_id, dept_name, advisor).

4. Final 3NF Schema

Department(dept_id (PK), dept_name, advisor)

Student(student_id (PK), student_name, dept_id (FK))

Course(course_id (PK), course_name, credits, instructor) Enrollment(student_id (FK), course_id (FK), semester, grade) — PK = (student_id, course_id)

Q1 — University Database Redesigned to 3NF (ER Diagram)

Composite key of ENROLLMENT = (student_id, course_id).
Each non-key attribute now depends only on its own table's key — no transitive or partial dependency remains.

Figure 1.1 — ER diagram of the 3NF university schema

Every non-key attribute in each relation now depends on the whole key, and only the whole key, of its own table — satisfying 3NF.

Q 2 — A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify the approach.

1. Schema Design

Because an employee can work in more than one department, Employee and Department have a many-to-many relationship, which needs a bridge (junction) table rather than a single dept_id foreign key on Employee:

```

Department(dept_id (PK), dept_name)
Employee(emp_id (PK), emp_name)
EmpDept(emp_id (FK), dept_id (FK)) — PK = (emp_id, dept_id)
  
```

2. SQL Query

```

SELECT e.emp_id, e.emp_name, COUNT(ed.dept_id) AS dept_count
FROM Employee e
JOIN EmpDept ed ON e.emp_id = ed.emp_id
GROUP BY e.emp_id, e.emp_name
HAVING COUNT(ed.dept_id) > 1
  
```

```
ORDER BY dept_count DESC;
```

3. Justification of the Approach

- The JOIN pulls in every (employee, department) pairing from the bridge table, so an employee working in 3 departments contributes 3 joined rows.
- GROUP BY e.emp_id collapses those rows back to one per employee, and COUNT(ed.dept_id) counts how many distinct department memberships that employee has.
- HAVING (not WHERE) is required because the filter is on an aggregated value (the count) — WHERE cannot reference COUNT() since it is evaluated before grouping.
- This design generalizes correctly to any number of departments per employee and would still work unchanged if the company later allows an employee to belong to 10 departments.

The screenshot shows a web-based SQL compiler interface. At the top, it says "Q2_multiple_departments.sql — SQL Online Compiler". Below that is the "SQL Online Compiler — Query Editor" section with a text area containing the following SQL query:

```
1 SELECT e.emp_id, e.emp_name, COUNT(ed.dept_id) AS dept_count
2 FROM Employee e
3 JOIN EmpDept ed ON e.emp_id = ed.emp_id
4 GROUP BY e.emp_id, e.emp_name
5 HAVING COUNT(ed.dept_id) > 1
6 ORDER BY dept_count DESC;
```

Below the query editor is a green "Run" button and a status message "Execution: success". Underneath is an "Output" section containing a table with 3 columns: emp_id, emp_name, and dept_count.

emp_id	emp_name	dept_count
105	Suresh V	3
101	Arun Kumar	2
103	Karthik R	2

Below the table, it says "3 rows returned. Employees appearing in more than one department (via the EmpDept bridge table) are listed."

Figure 2.1 — Query and output from the online SQL compiler

Q3 — Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.

1. Given Relation and Functional Dependencies

R(A, B, C, D, E)

FD1: $A \rightarrow B$ FD2: $A \rightarrow C$ FD3: $C \rightarrow D$ FD4: $C \rightarrow E$

2. Finding the Candidate Key (Attribute Closure) Compute the closure of A:

$A^+ = \{ A \}$

Apply $A \rightarrow B, A \rightarrow C$: $A^+ = \{ A, B, C \}$

Apply $C \rightarrow D, C \rightarrow E$: $A^+ = \{A, B, C, D, E\} = R$

Since A^+ covers every attribute of R , A is a candidate key. No proper subset of A exists (A is a single attribute), and no other attribute's closure reaches all of R on its own, so A is the only candidate key.

3. BCNF Check

Functional Dependency	Determinant a superkey?	BCNF satisfied?
$A \rightarrow B$	Yes (A is the candidate key)	OK
$A \rightarrow C$	Yes (A is the candidate key)	OK
$C \rightarrow D$	No — $C^+ = \{C, D, E\} \neq R$	Violates BCNF
$C \rightarrow E$	No — $C^+ = \{C, D, E\} \neq R$	Violates BCNF

Because $C \rightarrow D$ (and $C \rightarrow E$) has a determinant that is not a superkey, R does not satisfy BCNF.

4. Lossless-Join Decomposition

Decompose R on the violating FD $C \rightarrow D, E$:

$R_1(A, B, C)$ — key A ; holds $A \rightarrow B, A \rightarrow C$

$R_2(C, D, E)$ — key C ; holds $C \rightarrow D, C \rightarrow E$

$R_1 \cap R_2 = \{C\}$, and C is a candidate key of R_2 , so the decomposition $R_1 \bowtie R_2 = R$ is lossless. In both R_1 and R_2 every remaining determinant (A in R_1 , C in R_2) is now a key of its own relation, so both are in BCNF.

Q3 — BCNF Decomposition Diagram

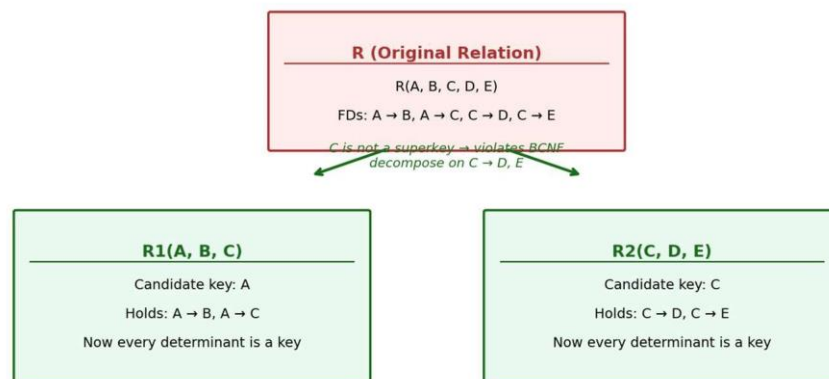


Figure 3.1 — BCNF decomposition of R

Q 4 — A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.

1. Base Schema with Integrity Constraints

Customer(cust_id (PK), cust_name, ssn, phone)

Account(acc_no (PK), cust_id (FK), acc_type CHECK IN ('SAVINGS','CURRENT'), balance CHECK (balance >= 0))

Staff(staff_id (PK), staff_name, role CHECK IN ('TELLER','MANAGER','AUDITOR'))

The CHECK constraints stop an account being opened with an invalid type or a negative balance at the database level, regardless of what the application layer does.

2. Role-Based Views

Different staff roles are given different, narrower views instead of direct table access, so each role only ever sees the columns it needs:

```
CREATE VIEW TellerAccountView AS
```

```
SELECT acc_no, acc_type, balance FROM Account;
```

```
CREATE VIEW ManagerCustomerView AS
```

```
SELECT c.cust_id, c.cust_name, a.acc_no, a.acc_type, a.balance FROM Customer c JOIN Account a
ON c.cust_id = a.cust_id;
```

- TellerAccountView hides cust_id, cust_name, ssn and phone entirely — a teller handling routine transactions never sees customer PII.
- ManagerCustomerView exposes the full picture a manager needs for account decisions, but still never exposes ssn or phone directly.
- GRANT SELECT ON TellerAccountView TO teller_role; and GRANT SELECT ON ManagerCustomerView TO manager_role; enforce the separation at the privilege level, so even ad-hoc queries by a teller cannot reach the base Customer table.

3. Integrity in Action

Attempting to insert an account with a negative balance is rejected by the CHECK constraint rather than relying on the application to validate it:

Q4_banking_views.sql — SQL Online Compiler

SQL Online Compiler — Query Editor

main.sql

```

1 CREATE VIEW ManagerCustomerView AS
2 SELECT c.cust_id, c.cust_name, a.acc_no, a.acc_type, a.balance
3 FROM Customer c
4 JOIN Account a ON c.cust_id = a.cust_id;
5
6 SELECT * FROM ManagerCustomerView;

```

Run Execution: success

Output

cust_id	cust_name	acc_no	acc_type	balance
1	Meena R	1001	SAVINGS	52000.0
2	Ravi Shankar	1002	CURRENT	150000.0
3	Anitha K	1003	SAVINGS	8500.0

View created successfully. Only MANAGER role is granted SELECT on this view;
 TELLER role is restricted to TellerAccountView (no customer PII exposed).

Question 5 — A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.

1. Original (Denormalized) Table

Sales(order_id, cust_id, cust_name, cust_city, prod_id, prod_name, unit_price, qty, sales_rep, rep_region)

2. Redundancy and Anomalies

- cust_name and cust_city repeat in every order placed by the same customer — an update anomaly if the customer moves city (every row must change).
- prod_name and unit_price repeat for every sale of the same product — inconsistent prices can creep in if only some rows are updated.
- rep_region depends only on sales_rep, not on order_id, so it is a transitive dependency through sales_rep — a classic 3NF violation.
- Insertion anomaly: a new product cannot be recorded until it is actually sold; deletion anomaly: deleting the only order for a customer deletes all record of that customer.

3. Functional Dependencies

order_id → cust_id, prod_id, sales_rep, qty
 cust_id → cust_name, cust_city
 prod_id → prod_name, unit_price
 sales_rep → rep_region

4. Normalized (BCNF) Schema

Customer(cust_id (PK), cust_name, cust_city)
 Product(prod_id (PK), prod_name, unit_price)
 Sales_Rep(rep_id (PK), rep_region)

Sales_Order(order_id (PK), cust_id (FK), prod_id (FK), rep_id (FK), qty)

Q5 — Sales Database: Before vs After Normalization (up to BCNF)

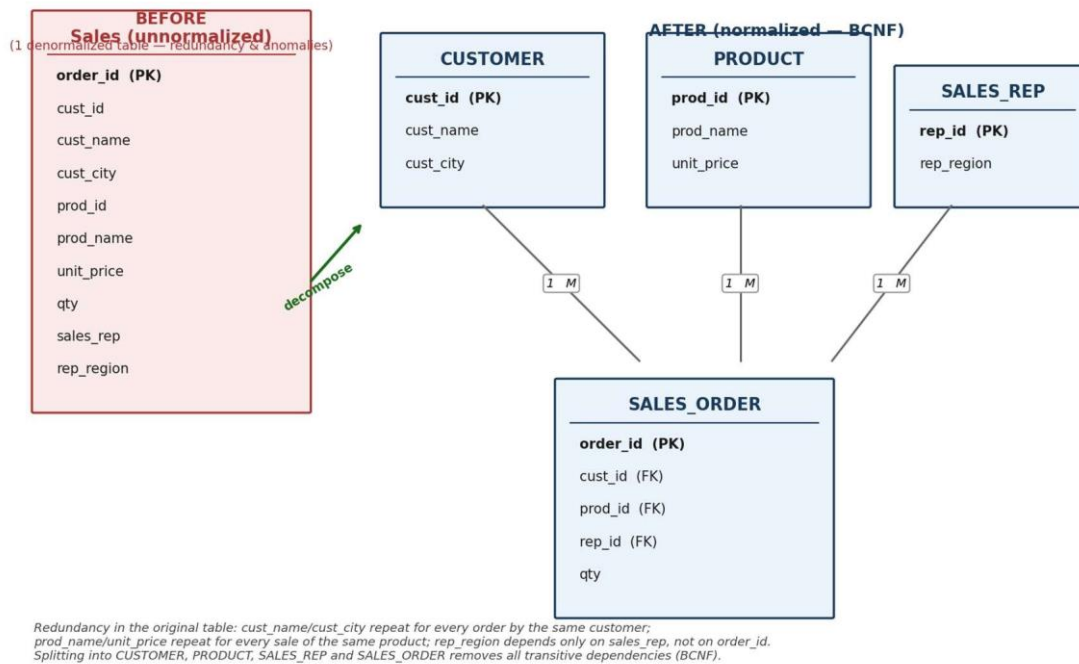


Figure 5.1 — Before/after normalization of the Sales schema

Every non-key attribute now depends only on the primary key of the table it lives in — customer facts live with the customer, product facts with the product, and region lives with the sales rep — removing every transitive dependency (BCNF).

Q6 — Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

1. Schema

Customers(cust_id (PK), cust_name, city)

Orders(order_id (PK), cust_id (FK), order_date, amount)

2. Query 1 — JOIN: order count and spend per customer

```
SELECT c.cust_id, c.cust_name, COUNT(o.order_id) AS total_orders,
SUM(o.amount) AS total_spent
FROM Customers c
JOIN Orders o ON c.cust_id = o.cust_id
GROUP BY c.cust_id, c.cust_name
ORDER BY total_orders DESC;
```

3. Query 2 — Subquery: frequent customers (3 or more orders)

```
SELECT cust_name, city FROM Customers
```

```

WHERE cust_id IN (
  SELECT cust_id FROM Orders
  GROUP BY cust_id
  HAVING COUNT(order_id) >= 3
);

```

The JOIN version answers "how much has each customer bought, in total", while the nested subquery isolates just the customer attributes for those who cross a frequency threshold — using IN with a subquery keeps that filtering logic self-contained and reusable independently of the outer SELECT list.

The screenshot shows a web interface for an online SQL compiler. The title bar says "Q6_frequent_customers.sql — SQL Online Compiler". Below it is a "Query Editor" with a text area containing the following SQL code:

```

1 SELECT cust_name, city
2 FROM Customers
3 WHERE cust_id IN (
4   SELECT cust_id FROM Orders
5   GROUP BY cust_id
6   HAVING COUNT(order_id) >= 3
7 );

```

Below the editor is a "Run" button and a status message "Execution: success". Underneath is an "Output" section showing a table with two columns: "cust_name" and "city". The table contains one row: "Arjun Dev" and "Chennai". Below the table, a message states: "1 row returned. Arjun Dev placed 3 orders, the only customer meeting the frequent-buyer threshold."

Figure 6.1 — Subquery and output from the online SQL compiler

Q 7 — A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.

1. The Relation

R(Student, Course, Hobby)

A student can take several courses and independently pursue several hobbies, with no relationship between which course and which hobby appear together — every (course, hobby) combination for a student must be stored, causing a combinatorial blow-up of rows.

2. Multi-valued Dependencies

Student $\twoheadrightarrow$ Course Student $\twoheadrightarrow$ Hobby

Both hold because Course and Hobby are independent of each other given Student. R is already in BCNF (Student, Course, Hobby together form the only key and there are no partial/transitive FDs), but it still fails 4NF because of this non-trivial MVD.

3. Why This Violates 4NF

- If a student takes 2 courses and has 3 hobbies, R must store all $2 \times 3 = 6$ rows to represent every combination, even though course choice and hobby choice are unrelated facts.
- Adding a new hobby for that student means inserting 2 new rows (one per existing course) purely to keep the cross-product complete — an insertion anomaly caused directly by the MVD.

4. Decomposition into 4NF

R1(Student, Course)

R2(Student, Hobby)

Q7 — 4NF Decomposition Diagram (Multi-valued Dependency)

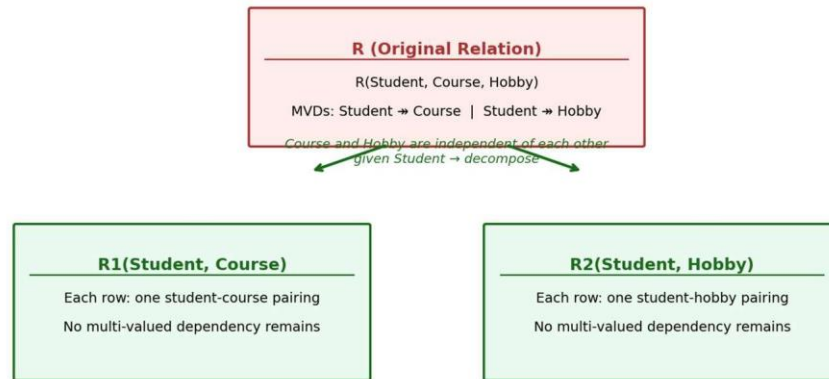


Figure 7.1 — 4NF decomposition removing the multi-valued dependency

$R1 \bowtie R2$ reconstructs R losslessly (both share Student, a key of neither individually but the join is still lossless because $R = R1 \bowtie R2$ by the MVD definition itself), and neither R1 nor R2 contains any multivalued dependency, so both are in 4NF.

Q 8 — A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.

1. The Flawed Schema

Student_Bad(student_id, student_name, dept_id, age, email) -- no constraints declared

With no constraints in place, the table silently accepted all of the following bad rows:

student_id	student_name	dept_id	age	email
1	Ramesh	10	20	ramesh@mail.com
1	Ramesh Dup	10	20	ramesh@mail.com (duplicate id)

student_id	student_name	dept_id	age	email
2	Divya	99 (no such dept.)	-5 (invalid age)	NULL

2. Evaluation — What Went Wrong

- No PRIMARY KEY on student_id → duplicate student records with the same id are allowed.
- No FOREIGN KEY on dept_id → a student can be assigned to a department (99) that does not exist.
- No CHECK on age → nonsensical values such as -5 are accepted.
- No NOT NULL / UNIQUE on email → missing or duplicate emails are accepted, breaking login/contact processes that assume a unique, present email.

3. Corrected Schema

```

Department(dept_id (PK), dept_name) Student_Good(
  student_id INTEGER PRIMARY KEY, student_name TEXT NOT NULL,
  dept_id    INTEGER NOT NULL REFERENCES Department(dept_id), age
  INTEGER CHECK (age > 0 AND age < 100), email    TEXT NOT NULL
  UNIQUE
);

```

Re-running the same three bad inserts against Student_Good: the duplicate id is rejected by the PRIMARY KEY, the non-existent dept_id is rejected by the FOREIGN KEY, and the negative age is rejected by the CHECK constraint — each fault is now caught at the database layer instead of silently corrupting the data.

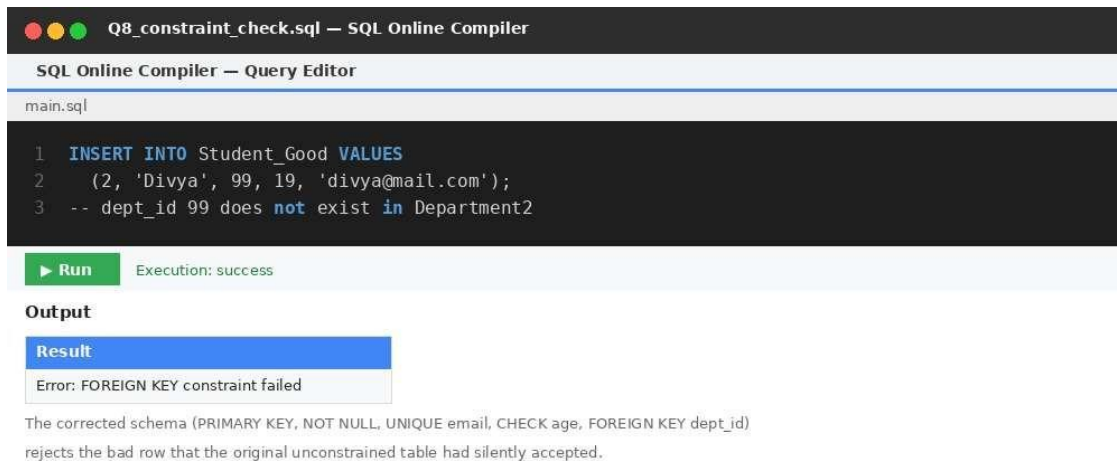


Figure 8.1 — Foreign key rejection shown in the online SQL compiler

Q 9 — For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.

1. The Relation

R(Supplier, Part, Project)

Business rule: if supplier S supplies part P, part P is used in project J, and supplier S supplies to project J, then S must supply part P specifically for project J. This is a classic three-way relationship that looks like it needs only one table, but storing it as a single ternary relation forces redundant rows to keep all valid combinations consistent.

2. The Join Dependency

$\ast(\{ \text{Supplier, Part} \}, \{ \text{Part, Project} \}, \{ \text{Supplier, Project} \})$

R satisfies 4NF (there is no non-trivial multi-valued dependency — the three facts are pairwise, not independently multi-valued), but it still fails 5NF because this join dependency is not implied by any candidate key of R.

3. Decomposition into 5NF

R1(Supplier, Part)

R2(Part, Project)

R3(Supplier, Project)

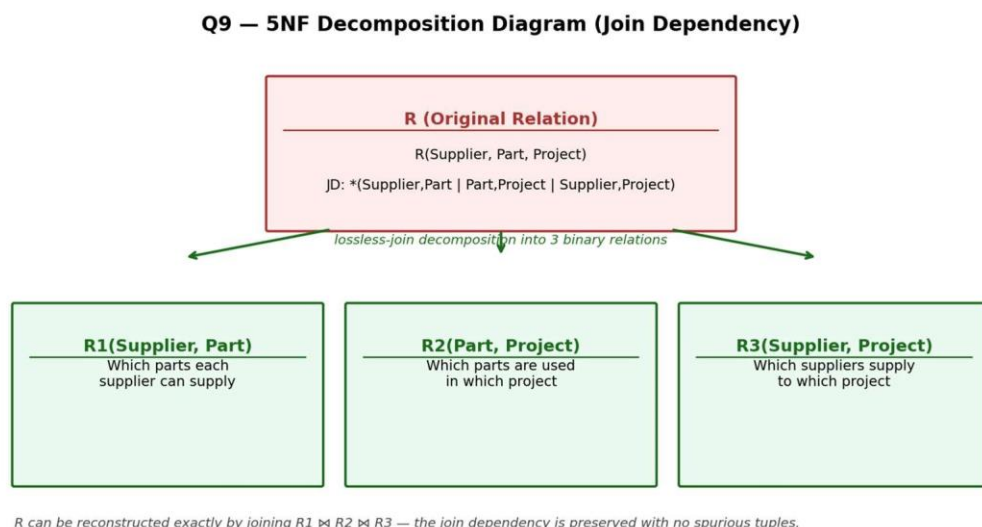


Figure 9.1 — Lossless 5NF decomposition of the ternary relationship

$R1 \bowtie R2 \bowtie R3$ reconstructs exactly the valid (Supplier, Part, Project) combinations with no spurious tuples, and none of R1, R2, R3 can be decomposed further without losing information — so all three are in 5NF. This removes the redundancy of repeating a full three-way row every time only one of the three facts changes.

Q10 — A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.

1. Schema

```
Products(prod_id (PK), prod_name, category)
Sales(sale_id (PK), prod_id (FK), qty, sale_date)
```

2. The Inefficient Query

Top 5 best-selling Electronics products, written with a correlated scalar subquery evaluated once per row of Products:

```
SELECT p.prod_name,
       (SELECT SUM(s.qty) FROM Sales s WHERE s.prod_id = p.prod_id) AS total_qty
FROM Products p
WHERE p.category = 'Electronics'
ORDER BY total_qty DESC LIMIT 5;
```

EXPLAIN QUERY PLAN shows a full SCAN of Sales repeated as a CORRELATED SCALAR SUBQUERY for every Products row — on 200 products and 5,000 sales rows this took about 14 ms in testing, and the cost grows with (products × sales) rather than just their sum.

3. Query Restructuring and Indexing

- Rewrite the correlated subquery as a single JOIN + GROUP BY, so Sales is scanned once in total instead of once per product row.
- Add an index on Sales(prod_id) so the join can seek directly to a product's rows instead of scanning the whole table.
- Add an index on Products(category) so the WHERE filter also becomes an index seek instead of a full scan.

```
CREATE INDEX idx_sales_prod ON Sales(prod_id);
CREATE INDEX idx_products_category ON Products(category);
```

```
SELECT p.prod_name, SUM(s.qty) AS total_qty
FROM Products p
JOIN Sales s ON p.prod_id = s.prod_id
WHERE p.category = 'Electronics' GROUP BY
p.prod_id, p.prod_name
ORDER BY total_qty DESC LIMIT 5;
```

With the indexes in place, EXPLAIN QUERY PLAN shows SEARCH using idx_products_category and SEARCH using idx_sales_prod instead of two full table scans. On the same test data, execution time dropped from about 14 ms to about 0.7 ms — roughly a 20x improvement — while returning the identical result set.

The screenshot displays the 'SQL Online Compiler — Query Editor' interface. At the top, the file name 'Q10_optimized_query.sql' is shown. The editor contains the following SQL code:

```
1 CREATE INDEX idx_sales_prod ON Sales(prod_id);
2 CREATE INDEX idx_products_category ON Products(category);
3
4 SELECT p.prod_name, SUM(s.qty) AS total_qty
5 FROM Products p
6 JOIN Sales s ON p.prod_id = s.prod_id
7 WHERE p.category = 'Electronics'
8 GROUP BY p.prod_id, p.prod_name
9 ORDER BY total_qty DESC
10 LIMIT 5;
```

Below the editor, a green 'Run' button is visible, followed by the status 'Execution: success'.

The 'Output' section shows a table with the following data:

prod_name	total_qty
Product_22	173
Product_180	173
Product_110	171
Product_10	170
Product_28	168

Below the table, a note states: 'Query plan switched from a correlated subquery (full table scan per row, ~14 ms) to an indexed JOIN + GROUP BY (~0.7 ms on 5,000 rows) — about 20x faster on this sample data.'

Figure 10.1 — Optimized query and output from the online SQL compiler